

AI Parts Wizard — Specification

AI-Driven Part Standardization & Component Rationalization Engine

Status: v1.0 — all 8 phases built · Owner: Darrin Weber · Last updated: 2026-10-05
Target: any Databricks workspace with Unity Catalog, serverless, Lakebase and Apps — see [DEPLOY.md](#)

1. Overview & Problem Statement

1.1 Background

A regional manufacturing ecosystem operates multiple specialized design centers that produce heavy machinery — e.g., tractors in one facility, harvesters in another. Over years of independent product design, separate engineering teams have created **identical physical components under completely different part numbers**.

Because each design center evolved its own catalog and numbering conventions, the organization now:

- Purchases the **same physical part** from different suppliers at conflicting price points, varying material qualities, and inconsistent lead times.
- Cannot manually reconcile catalogs — the volume of parts (~200K) far exceeds what human engineers can review by hand in spreadsheets.
- Loses negotiating leverage and carries redundant inventory because duplicates are invisible across facilities.

1.2 Goal

Build an automated, AI-powered system on the **Databricks Data Intelligence Platform** that reads part attributes, calculates physical and material similarity scores, and highlights identical or near-identical parts for procurement consolidation — so the organization can standardize on normalized part numbers and negotiate from a position of consolidated volume.

1.3 Target Users

Persona	Needs
Procurement analyst	See flagged duplicate clusters ranked by savings potential; compare supplier pricing/lead times; drive consolidation.
Plant / engineering manager	Review and disposition part-match groups; assign normalized part numbers; track progress.
Non-technical engineering lead	Ask natural-language questions about parts, similarity, and suppliers without writing SQL.

1.4 Success Metrics

Two tiers. Targets are **illustrative** for the demo (synthetic data), not commitments.

Business outcomes — does it save money?

- **\$ savings identified** via consolidated purchasing across matched clusters.
- **# of duplicate part groups** detected and dispositioned.
- **% of catalog rationalized** (parts assigned to a normalized enterprise part number).
- **Reduction in supplier count / SKU count** for equivalent components.

Engine quality — can the matching be trusted? Measured against the §4 seeded ground truth:

- **Precision / recall / F1** of duplicate detection (true-positive duplicates found vs. functional hard negatives correctly rejected).
- Reported per category (§9.5). **Measured (Phase 4):** overall recall 99.7%, precision ~100%, F1 99.9% on the labeled set; persisted to `gold_eval_metrics`.

2. Solution Summary

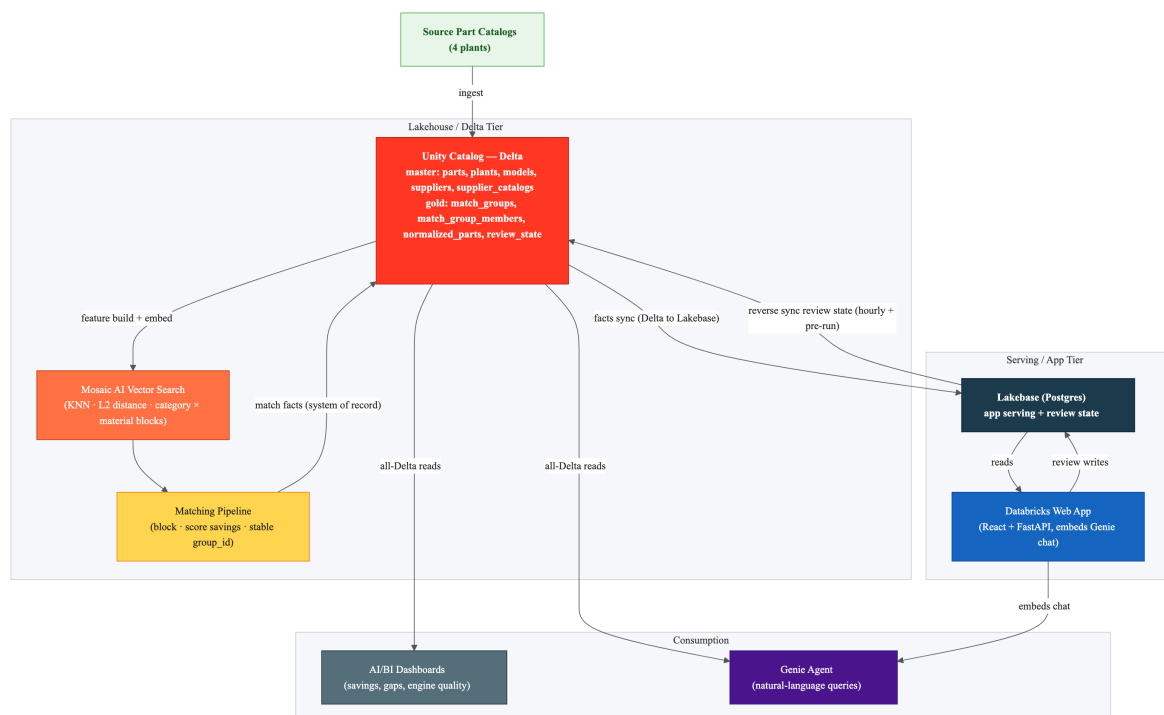
An **AI-Driven Part Standardization & Component Rationalization Engine** on the Databricks Lakehouse that automates part comparisons across design centers. Three pillars:

1. **Centralized Multi-Factory Data Cataloging** — Structured part specifications (weight, material density, physical dimensions, supplier codes, unit costs) from all four plant locations, governed in **Unity Catalog**.

2. **AI Similarity Scoring & Vector Search — Mosaic AI Vector Search** builds multidimensional embeddings per part from its physical characteristics, then retrieves nearest neighbors via **K-Nearest Neighbors on Euclidean (L2) distance** within `(category, material)` blocks (see §9 for why L2 over cosine). The engine compares parts across catalogs and outputs a similarity ranking that flags duplicates.
3. **Conversational Exploration & BI Visualization —** Interactive **AI/BI dashboards** over Databricks SQL surface flagged clusters and supplier price gaps; **Databricks Genie** lets non-technical leads ask natural-language questions such as:

"Show all harvester parts with greater than 95% material similarity to tractor part X that have lower supplier lead times."

2.1 Architecture at a Glance



Delta is the system of record for match facts; Lakebase serves the app and owns review state. Facts sync Delta→Lakebase; review state reverse-syncs Lakebase→Delta (hourly + before the nightly run). Dashboards and Genie read the all-Delta join — no federation. See §3.3.

3. Data Model

3.1 Identity & hierarchy

Parts exist within a strict **Plant** → **Model** → **Part** hierarchy, with a separate, side-by-side **normalized enterprise part number** layered on during review.

Native identity — global surrogate keys. Every part gets an opaque, globally unique surrogate `part_id`. The key encodes no meaning: the hierarchy travels as attributes (`model_id` on the part; `plant_id` on the model), and similarity is computed on the part's *feature vector*, never on its key. This keeps the vector index and every match table referencing a single clean column, and leaves cross-model comparison fully open — a match group can contain parts from any mix of models and plants.

- The surrogate must be **stable across pipeline reruns** (assigned once and persisted, or derived deterministically from a stable natural key) — not a volatile `monotonically_increasing_id()`.
- `plant_id` is **derived through the model**, not stored redundantly on the part.

Local numbering — unique within a model. The original plant-local `part_number` is retained as an attribute but is only unique *within a model*. The same string can recur across different models/plants and mean different physical parts. This reflects the independent design centers and enforces a hard rule:

Join-safety rule: never join, group, or dedupe on `part_number` alone. All cross-part operations key off the surrogate `part_id` (or the full hierarchy).

Normalized enterprise part number — assigned in review. A distinct identity that does *not* exist at ingest. When the engine matches similar parts and a reviewer confirms the group, the reviewer assigns one canonical enterprise part number to the cluster. Modeled as its own entity (`normalized_parts`) with a nullable `normalized_part_id` back-reference on each part — so a part can be unassigned, or belong to exactly one normalized part once rationalized.

3.2 Master data (Delta tables in Unity Catalog)

Approximate scale:

Entity	Volume	Notes
Plants	4	Physical design/manufacturing centers.
Models / Designs	Dozens	Harvesters and Tractors.
Parts	~200K	The core rationalization entity.
Suppliers	100s	Vendors supplying parts.
Supplier Catalogs	1 per supplier	Price per unit, unit discount, lead time, etc.

Catalog layout: the target workspace does **not** permit creating a new catalog, so all objects live in an existing `<catalog>` under a new schema

`ai_parts_wizard` — i.e. `<catalog>.ai_parts_wizard.<table>`. Because the medallion layers cannot each be their own schema, they are expressed as **table-name prefixes** within that single schema: `bronze_*` (raw ingest), `silver_*` (cleaned/standardized), `gold_*` (serving/analytics) — e.g. `silver_parts`, `gold_match_groups`.

Table sketches (per-category columns defined in §3.2.1). The `parts` table is a **wide table with nullable dimensional columns** — common attributes are typed columns; category-specific dimensions are nullable and populated only where they apply (defined in §3.2.1):

- `plants` — `plant_id` (PK), `plant_name`, `location`, `machine_focus` (tractor/harvester/both).
- `models` — `model_id` (PK), `plant_id` (FK), `machine_type` (Harvester | Tractor), `model_name`.
- `parts` — `part_id` (PK, surrogate), `model_id` (FK — plant derived via model), `part_number` (local, unique within model), `part_category` (one of the 8 in §3.2.1), `material`, `material_density`, `weight`, `description` (human-readable summary — display & Genie only, **not** a matching feature), plus the nullable dimensional & functional columns of §3.2.1 (populated only for the categories that use them), `normalized_part_id` (FK, nullable — assigned in review), `annual_volume` (units purchased per year — drives annual spend and savings).
- `normalized_parts` — `normalized_part_id` (PK), `enterprise_part_number`, `part_category`, `assigned_by`, `assigned_at`, `notes`.
- `suppliers` — `supplier_id` (PK), `supplier_name`, `region`.
- `supplier_catalogs` — `supplier_id` (FK), `part_id` (FK), `price_per_unit`, `unit_discount`, `lead_time_days`, `currency` (ISO code; demo data is USD), `material_quality_grade`.

3.2.1 Part categories & attributes

The demo models **8 part categories**, chosen to span distinct geometry archetypes so the engine is exercised across genuinely different dimensional schemas. Every part also carries the universal features `weight` and `material_density`; `part_category` and `material` are blocking keys (§9.2), not vector features.

Category	Geometry archetype	Dimensional columns	Functional discriminators
Bolt	solid cylinder + thread	<code>nominal_diameter</code> , <code>length</code> , <code>head_width</code>	<code>thread_pitch</code> , <code>tensile_strength</code>
Pin / Shaft	solid cylinder	<code>diameter</code> , <code>length</code>	<code>hardness</code>
Hose	hollow cylinder	<code>inner_diameter</code> , <code>outer_diameter</code> , <code>length</code>	<code>pressure_rating</code>
Bushing / Bearing	annular	<code>inner_diameter</code> , <code>outer_diameter</code> , <code>width</code>	<code>load_rating</code>
Gasket	annular / flat ring	<code>inner_diameter</code> , <code>outer_diameter</code> , <code>thickness</code>	<code>temperature_rating</code>
Bracket / Plate	prismatic block	<code>length</code> , <code>width</code> , <code>thickness</code>	<code>hole_count</code>
Wire / Cable	linear	<code>length</code> , <code>gauge</code>	<code>voltage_rating</code> , <code>conductor_count</code>
Seal / O-ring	annular fine	<code>inner_diameter</code> , <code>cross_section_diameter</code>	<code>hardness</code>

Canonical nullable column union on `parts` (each populated only for its categories): `length`, `width`, `thickness`, `diameter`, `inner_diameter`, `outer_diameter`, `nominal_diameter`, `cross_section_diameter`, `head_width`, `gauge`, `thread_pitch`, `tensile_strength`, `hardness`, `pressure_rating`, `load_rating`, `temperature_rating`, `hole_count`, `voltage_rating`, `conductor_count`.

Functional discriminators keep look-alike-but-not-interchangeable parts apart (two bolts of equal size but different `thread_pitch` are *not* duplicates). Note `hardness` appears in two categories on different scales (Shore A for seals, HRC for pins) — fine, because features are normalized **per category** (§9.4), so scales never cross blocks.

`description` is a generated human-readable summary (e.g. "Steel Bolt, M12 x 50 mm, pitch 1.5 mm") for the app UI and Genie. It is **not** part of the similarity vector (§9.4).

3.3 Match results & application data

Match results and review state are **two datasets with two systems of record**. Delta owns *what the engine found*; Lakebase owns *what humans decided*. Neither owns the other.

Dataset	Sys-tem of record	Flow
Match facts (<code>match_groups</code> , <code>match_group_members</code> , <code>savings</code>)	Delta	Machine-generated by the pipeline; synced one-way Delta → Lakebase for app reads.
Review state (dispositions, assigned normalized part #, notes)	Lakebase	Human-generated in the app; reverse-synced Lakebase → Delta for dashboards.

Delta — match facts (SoR):

- `match_groups` — a cluster of similar parts + rollup metrics (savings potential, member count, and **span metadata**: distinct models / machine types / plants the members span, powering the §6.2 duplication-span filters; generated-at), keyed by a **stable, content-derived** `group_id` (see below).
- `match_group_members` — parts belonging to a group; `part_id` + per-part similarity score. Retains `model_id` and `machine_type` as **retrievable metadata** so cross-model filters (e.g., harvester ↔ tractor only) apply without extra joins.

Lakebase — app serving + review state (SoR for review):

- Synced (read-only) copies of `match_groups` / `match_group_members` for low-latency app reads.
- `review_dispositions` — status per group (`new` , `reviewed` , `consolidated` , `dismissed`), reviewer, timestamp, notes — keyed by the pipeline's stable `group_id` .
- Normalized-part assignments written during review.

Stable `group_id` — **survives pipeline reruns**. The matching pipeline (§5) assigns each group a **deterministic, content-derived** `group_id` (e.g., derived from the sorted set of member `part_id` s) and **reuses it across runs**

when the group's membership is unchanged. Dispositions and normalized assignments key off this `group_id`, so a group marked `reviewed` yesterday stays reviewed after today's rerun and does not resurface. Stability is owned by the job, not the app layer.

Reverse sync — Lakebase review state → Delta (single source for all readers). A scheduled **batch** job MERGES the Lakebase-owned review state (dispositions + normalized assignments) into a **Delta gold table**. This Delta copy is the **single source both dashboards and the matching pipeline read** — there is **no query federation** anywhere in the design. The Delta `normalized_parts` table and the `parts.normalized_part_id` back-reference are populated from this reverse sync rather than edited in place by the app.

Cadence:

- **Hourly** on a schedule, to keep dashboards (§7) reasonably current (staleness $\leq \sim 1$ hour).
- **Immediately before the nightly matching job**, so the pipeline's suppression step (§5) sees the latest dispositions. Ordered as a dependency: **reverse-sync** → **matching job**.

App writes still take effect immediately in the app itself (it reads Lakebase live); the reverse sync only governs how quickly decisions reach Delta-based dashboards and the pipeline.

Sync mechanism (as built). Both directions are implemented as Spark **notebooks** using the native `postgresql` data source, rather than Lakebase managed **synced tables**:

- *Reverse (Lakebase → Delta)* — a notebook is required regardless: synced tables only flow Delta → Lakebase, so there is no managed option for this direction.
- *Forward (Delta → Lakebase)* — a notebook was chosen so the serving copies can be **denormalized** (members joined with part details + best price; offers joined with supplier names), which a 1:1 synced table can't do, and to keep one uniform mechanism for both directions. Trade-off: we forgo synced tables' managed incremental/CDC refresh. **Future refinement:** build denormalized Delta gold tables (CDF-enabled) and point managed synced tables at those for the forward direction, keeping the notebook only for the reverse.

4. Sample Data Generation

We need a **large synthetic dataset** to demonstrate the engine at realistic scale ($\sim 200K$ parts) with *planted duplicates* — parts that are intentionally

identical or near-identical across the tractor and harvester catalogs so the matching engine has true positives to find.

- **Approach:** Databricks notebooks (Python) generating parts, suppliers, catalogs.
 - **Realism requirements:**
 - The **8 part categories** of §3.2.1, each generated with its own dimensional columns, functional discriminators, and plausible materials.
 - **USD pricing at typical market levels** — a per-category list-price range (e.g. bolts ~\$0.30–9, seals ~\$0.60–28, hoses ~\$18–250, brackets ~\$12–420), scaled log-wise by part size and by a material multiplier (e.g. titanium ×4, stainless ×1.6), then ±20% per supplier offer so savings deltas are visible.
 - **Annual purchase volume** per part — log-uniform per category (bolts/seals 20–2,000 units/yr, gaskets 10–600, other categories 5–400) — so savings are expressed per year.
 - English supplier names and regions (`Supplier 0` ... `Supplier 199`).
 - **Seeded ground truth** (we control it, so §9.5's per-category thresholds have labels to tune against):
 - **True-positive duplicates** — take a part and re-emit it in another model/plant under a different `part_number` , keeping category + material + dimensions + functional discriminators the same (optionally within tiny manufacturing noise), at a different supplier price/lead time. These *should* match.
 - **Functional hard negatives** — parts with identical dimensions but a differing functional discriminator (e.g., same-size bolts with different `thread_pitch` , or hoses with different `pressure_rating`). These *should not* match, and exist to prove the functional discriminators earn their place (§9.4).
 - **Cross-material near-misses** — identical geometry, different `material` . Excluded by blocking (§9.2); included to confirm blocking works.
 - A **ground-truth label table** recording each seeded relationship (duplicate / hard negative / near-miss) for precision/recall measurement.
 - **Tooling candidates:** Polars + Mimesis for local/UC generation, or dbldatagen for Connect/notebook-scale generation.
 - **Deliverable:** a repeatable generation notebook/job + the ground-truth label table.
-

5. Scheduled Job + Matching Pipeline

A scheduled Databricks Job (Lakeflow pipeline) that:

1. **Builds/refreshes the feature table first.** Computes per-category normalization stats (`gold_feature_stats`) and applies them to produce one **canonical normalized feature vector per part** (`gold_part_features`). This table is the single shared artifact both the batch matcher and the online Vector Search index read (§9.1).
2. Runs the **batch matcher in Spark** — KNN via LSH `approxSimilarityJoin` on **Euclidean (L2) distance within** (`category`, `material`) **blocks** (§9.2–§9.3) — and clusters the near-neighbor pairs into groups.
3. Assigns each group a **deterministic, content-derived** `group_id` (from the sorted set of member `part_id` s) and reuses it across runs when membership is unchanged — so review state stays attached to the same group (§3.3). Stability is the job's responsibility.
4. Computes **annual savings potential** per group: for each member, (its current best unit price – the best unit price available across the group) × its `annual_volume` , summed over the group. Annual spend is $\sum$ member best price × annual volume.
5. Writes match facts (`gold_match_groups` , `gold_match_group_members`) to **Delta** (system of record), which then sync to **Lakebase** to serve the app (§3.3).
6. Joins in current dispositions from the **reverse-synced Delta gold table** (§3.3) — no federation — so groups already marked `reviewed` / `dismissed` are de-prioritized and do not resurface.

The **online path** (app "find similar", Genie) uses a **Mosaic AI Vector Search** index that is a Delta Sync off `gold_part_features` — the *same* vectors — so batch and online stay consistent (§9.1). The batch groups are authoritative for the review workflow; Vector Search is the live per-part lookup.

Nightly sequence: a reverse-sync of review state runs **immediately before** this job (§3.3), guaranteeing the suppression step reads the latest dispositions.

Cadence: nightly full rebuild. Each night the job rebuilds embeddings and re-matches all blocks — simple, predictable, and always ≤ 1 day fresh, which suits a relatively stable manufacturing catalog. (Incremental, change-triggered rebuilds are a future optimization if catalog churn or compute cost warrants it.)

6. Databricks Web App

Stack: React front end + FastAPI back end + Lakebase, deployed as a Databricks App.

Deployed (Phase 6): `app ai-parts-wizard` — reads/writes Lakebase `apw` as its service principal. Implementation in `code/app/`.

6.1 Core capabilities

- **Review & visualize** part rationalization results.
- **Rank by potential savings** — highest-impact match groups first.
- **Disposition matches** — mark a group as reviewed/other status so it stops resurfacing once handled.
- **Assign a normalized part number** to a group of similar parts.
- **Supplier pricing deltas** — show price/lead-time gaps across suppliers for parts in a group.
- **Chat window** — **global, app-level** natural-language interaction over the *entire* catalog (parts, members, suppliers, match groups), Genie-backed (§8) — e.g. "show all harvester parts >95% similar to tractor part X". It is **not** scoped to a single group. Placement: a **panel docked at the bottom of the Match Queue (parts list)**, always available while browsing. (An optional per-group shortcut may pre-seed a group-scoped question, but the primary chat is catalog-wide.) *Phase 6 ships a disabled stub in the group drawer; Phase 7 relocates it to the queue-bottom panel and wires Genie.*

6.2 Filtering

All filters AND-combine. Grouped by what they slice:

What the group is about - **Part category** (one of the 8 — every group is single-category by blocking, §9.2) - **Material** (single-material by blocking) - **Group size** — number of member parts

Duplication span — how far apart the duplicated parts sit in the hierarchy. Each match group carries computed span metadata (distinct **models**, **machine types**, and **plants** its members touch, §3.3), enabling: - **Spans > 1 model** — catches cross-model duplication (incl. model-version to model-version), the highest-*volume* source. - **Spans > 1 machine type** — Harvester ↔ Tractor cross-type duplication. - **Spans > 1 plant** — cross-design-center duplication, often the highest *value* (different suppliers/contracts).

Where the parts come from - **Model · Plant · Machine type** (Harvester | Tractor) · **Supplier** (groups where a supplier appears)

Business value / confidence - Annual savings potential — min threshold + the default sort - **Similarity score** — min 0–1 similarity (hide weak matches)

Workflow state - Review status — new / reviewed / consolidated / dismissed - **Normalized?** — whether an enterprise part number has been assigned

Default view. On open, the Match Queue defaults to **review status = new AND spans > 1 model**, sorted by **savings descending** — "your highest-value untouched cross-context duplicates." This surfaces all cross-model, cross-type, and cross-plant duplication while hiding only trivial intra-model dupes; reviewers can clear the span default to see those too.

6.3 Suggested primary views (to validate)

- **Match Queue** — prioritized list of groups by savings, with status chips; a **global Genie chat panel docked at the bottom of this parts list** (always available, catalog-wide — §6.1/§8), so users can ask questions while browsing the queue.
- **Group Detail** — member parts, per-part similarity, supplier pricing table, normalize/disposition actions.

7. Dashboards & Visualizations

Deployed (Phase 8): Lakeview dashboard ("AI Parts Wizard — Rationalization"), published on the project SQL warehouse. Definition in `code/dashboards/` .

AI/BI (Lakeview) dashboards over Databricks SQL, reading the **all-Delta** join of match facts + reverse-synced review state (§3.3) — no federation. Two lenses:

Business benefit

- **Annual savings potential (USD)** — total, and broken out by category, plant, and **duplication span** (cross-plant vs. cross-type vs. cross-model), since cross-plant dupes typically carry the highest value.
- **Supplier price-gap analysis** for matched clusters (price/lead-time deltas).
- **Rationalization progress** — groups by disposition status (new/reviewed/consolidated/ dismissed) and catalog % normalized over time.
- **Duplicate concentration** — by category, machine type, and span, to show where the redundancy lives.

Engine quality (proves the matches are trustworthy, §1.4)

- **Precision / recall / F1 per category** against the §4 seeded ground truth.
- Distribution of similarity scores and where the per-category thresholds (§9.5) sit.

8. Genie Agent

Deployed (Phase 7): Genie space ("AI Parts Wizard") over the gold/silver tables. Wired into the app's docked **Ask Genie** panel (§6.1) via a `/api/chat` proxy to the Genie Conversation API, run as the app service principal. Validated end to end (e.g. "total annual savings potential" → \$36,112,562.69). Definition in `code/genie/`.

- **Natural-language inquiries** over the parts / match / supplier data (the all-Delta facts + review state of §3.3).
- **Embedded** in the web app chat window (§6.1) — a global, catalog-wide panel docked at the bottom of the Match Queue.
- **Potentially accessible outside** the app as well (standalone Genie space).
- Example prompt:

"Show all harvester parts with greater than 95% material similarity to tractor part X that have lower supplier lead times."

The "95% similarity" in that prompt maps directly to the **normalized 0–1 similarity score** (§9.5) — the engine, app filters, and Genie deliberately share that one language.

Requires a curated Genie space with well-described tables and metrics so NL maps cleanly: `parts` (+ categories/attributes, §3.2.1), `match_groups` (savings + span metadata), `match_group_members` (similarity scores), `supplier_catalogs` (price/lead-time), and `normalized_parts`. Seed it with example queries covering savings, span, and similarity.

9. Vector DB & Matching Parameters

9.1 Engine — one shared feature vector, two engines

Each part has **one canonical normalized feature vector**, computed once by the pipeline and persisted in `gold_part_features` (§5). Two engines read that same vector, using **Euclidean (L2)** distance (see §9.3 for why, over cosine):

- **Batch matcher (pipeline, Spark):** builds the authoritative `gold_match_groups` for the review workflow via LSH `approxSimilarityJoin` (KNN by L2 within blocks). Right tool for a scheduled all-catalog pass.
- **Online lookups (Mosaic AI Vector Search):** a Delta Sync index over `gold_part_features` serves the app's interactive "find similar" and Genie — live per-part nearest-neighbor queries with `(category, material)` metadata filters.

Because both read the *same* stored vector, use the *same* metric, and apply the *same* per-category thresholds, they agree for the near-duplicate cases that matter. Both are approximate NN, so they can differ on borderline pairs — which is why the **batch groups are the system of record** and Vector Search is the exploratory live lookup. Embeddings are multidimensional representations of each part's physical characteristics (§9.4).

9.2 Blocking — compare within (category + material)

Parts are only compared within a **block keyed by** `(part_category, material)`. A steel bolt is scored only against other steel bolts, never against aluminum bolts or hoses. This delivers three things at once:

- **Homogeneous feature vectors.** Within a category, every part shares the same dimensional schema, so the nullable per-category columns (§3.2.1) are all populated and directly comparable — the heterogeneous-geometry problem disappears.
- **No cross-material false matches.** Identical dimensions in a different material are a different procurable part; blocking on material excludes them by construction.
- **Smaller search space.** A part is never scored against the ~200K parts outside its block.

Category and material therefore act as **hard blocking keys**, not soft vector features.

9.3 Distance metric — Euclidean (L2), not cosine

The source narrative suggested cosine similarity, but cosine is **scale-invariant**: two parts with proportional dimensions (e.g., every dimension 2×) point in the same direction and score as near-identical, even though they are physically different parts. For duplicate detection, **absolute size is the signal**, so we use **Euclidean (L2) distance on normalized features**, which Mosaic AI Vector Search supports.

9.4 Feature vector & normalization

Within a block, the vector is built from that category's attributes as defined in §3.2.1:

- **Category-relevant dimensions** (e.g. `nominal_diameter`, `length`, `head_width` for bolts; `inner_diameter`, `outer_diameter`, `length` for hoses).
- **Functional discriminators** for the category (e.g. `thread_pitch` + `tensile_strength` for bolts, `pressure_rating` for hoses) — these keep look-alike-but-not-interchangeable parts from being scored as duplicates.
- **Weight** and **material density** (universal).

(`part_category` and `material` are blocking keys, not vector dimensions — §9.2.)

Numeric features are **normalized per category** (e.g., z-score within the block) so no single attribute dominates the L2 distance and each category's distribution is respected. This per-category normalization also means attributes measured on different scales across categories (e.g. `hardness` in Shore A vs HRC) never interact — each block is normalized independently.

9.5 Duplicate flagging — hybrid: per-category threshold + top-K floor

Flagging combines a tuned threshold with a KNN safety net, and reports an interpretable score:

1. **Top-K retrieval.** Within each `(category, material)` block, retrieve each part's **top-K nearest neighbors** by L2. This is native to KNN and guarantees a non-empty candidate set per part even if nothing clears the threshold.
2. **Normalized 0–1 similarity.** Convert per-category L2 distance into a **0–1 similarity score** (percentile within the category, or $1/(1+d)$), so thresholds and the app UI read as "≥ 95% material similarity" — matching the Genie example prompt in the source.

3. **Per-category thresholds, tuned on ground truth.** Each category gets its own thresholds, tuned against the **§4 seeded duplicates** (which we control, so labeled duplicates per category are guaranteed):
4. **$\geq$ high threshold** → strong duplicate, auto-grouped.
5. **middle band** → routed to human review.
6. **below** → dropped.

Rationale: per-category tuning reflects that tolerance differs by category (fasteners are tightly standardized; hoses/gaskets vary more); the top-K floor keeps the review queue from starving on a miscalibrated cutoff; the normalized score keeps the engine, app, and Genie speaking the same "% similarity" language.

Tuned (Phase 4, measured on §4 ground truth): duplicate-pair L2 tops out at 0.093 and hard-negatives sit at $L2 \geq \sim 1.0$, a clean gap — so a single global `L2_THRESHOLD = 0.12` yields **recall 99.7%, precision ~100%, F1 99.9%** on the labeled set across all 8 categories. Per-category thresholds are therefore not required for accuracy.

Coincidental near-duplicates: the matcher also groups unlabeled parts that are genuinely near-identical (concentrated in low-attribute categories like Pin/Shaft and Seal/O-ring, whose small feature space makes coincidence common). These are treated as **legitimate additional consolidation candidates**, not false positives; tightening the threshold to suppress them would discard real near-duplicates. Metrics persist to `gold_eval_metrics` for the §7 engine-quality dashboard.

10. Cross-Cutting Concerns

- **Governance & security:** Unity Catalog governs the Delta data; app auth via Databricks App identity; Lakebase access controls; the Genie space (§8) scoped to the same governed tables. The sync notebooks connect to Lakebase with a short-lived OAuth token; in the demo it is passed as a run-time job parameter, but production should source it from a Databricks secret / the job's service-principal identity rather than a plaintext parameter.
- **Evaluation:** precision/recall/F1 per category against the §4 seeded ground truth — now a first-class success metric (§1.4), surfaced on the engine-quality dashboard (§7).

- **Data freshness:** nightly full rebuild of embeddings + matches (§5); review state reaches dashboards hourly and the pipeline pre-run via reverse sync (§3.3); app sees its own writes live.
 - **Scalability:** ~200K parts, but `(category, material)` blocking (§9.2) keeps each KNN comparison to a small block rather than the full catalog — the key scale lever.
 - **Identity stability:** surrogate `part_id` and content-derived `group_id` (§3.1, §3.3) keep parts and match groups stable across nightly reruns so review state never orphans.
-

11. Open Questions & Decisions

Decided

- **✓ Part identity:** global surrogate `part_id` (opaque, stable across reruns); hierarchy travels as attributes. (§3.1)
- **✓ Local numbering:** `part_number` unique *within a model*; never join/group on it alone. (§3.1)
- **✓ Normalized enterprise part number:** side-by-side identity assigned during review via `normalized_parts`. (§3.1)
- **✓ Parts schema shape:** wide table with nullable per-category dimensional columns. (§3.2, §9.2)
- **✓ Match results system-of-record:** two SoRs — Delta owns match facts, Lakebase owns review state. (§3.3)
- **✓ Facts → app:** match facts sync one-way Delta → Lakebase for low-latency reads. (§3.3)
- **✓ Review state → dashboards:** reverse sync Lakebase → Delta gold; dashboards read all-Delta. (§3.3)
- **✓ Disposition stability:** dispositions key off a deterministic, content-derived `group_id` the pipeline reuses across reruns. (§3.3, §5)
- **✓ Reverse sync (Option B — copy, no federation):** scheduled **batch MERGE, hourly** for dashboards + a run **immediately before** the nightly matching job; both dashboards and the pipeline read the Delta copy. (§3.3, §5)
- **✓ Blocking:** compare only within `(part_category, material)` blocks; category and material are hard blocking keys, not vector features. (§9.2)
- **✓ Distance metric:** Euclidean (L2) on normalized features, not cosine (scale matters for physical duplicates). (§9.3)

- **✓ Feature vector & scaling:** category-relevant dimensions + weight + density, normalized per category (z-score). (§9.4)
 - **✓ Duplicate flagging:** hybrid — top-K KNN floor + per-category thresholds tuned on §4 ground truth, reported as a normalized 0–1 similarity. (§9.5)
 - **✓ Part categories & attributes:** 8 categories with per-category dimensional columns + functional discriminators; universal weight/density; category/material as blocking keys. (§3.2.1)
 - **✓ Sample-data ground truth:** seed true-positive duplicates, functional hard negatives, and cross-material near-misses + a label table. (§4)
 - **✓ App filters:** full set + duplication-span filters (spans >1 model / type / plant); default view = unreviewed + spans >1 model, sorted by savings. Groups carry span metadata. (§6.2, §3.3)
 - **✓ Pipeline cadence:** nightly full rebuild; reverse-sync runs immediately before. (§5)
 - **✓ Success metrics:** two tiers — business outcomes + engine quality (precision/recall/F1 vs. §4 ground truth, per category); targets illustrative. (§1.4)
 - **✓ Build-time tuning (Phase 4):** measured duplicate L2 max = 0.093, hard-negative $\geq \sim 1.0$; global `L2_THRESHOLD = 0.12` → recall 99.7% / precision $\sim 100\%$ / F1 99.9% on the labeled set; per-category thresholds not needed. Coincidental near-duplicates treated as legitimate candidates. Metrics in `gold_eval_metrics`. (§9.5, §1.4)
-

12. Implementation Plan

12.1 Target environment

Everything is built and deployed in a single Databricks workspace (the reference build ran on AWS us-east-1); `DEPLOY.md` automates it. All assets live there:

- **Unity Catalog:** existing catalog `<catalog>`, new schema `ai_parts_wizard` (no catalog-create permission); medallion layers as `bronze_*` / `silver_*` / `gold_*` table-name prefixes (§3.2).
- **Lakebase:** a Postgres instance in the workspace for app serving + review state (§3.3).
- **Compute:** a SQL warehouse (dashboards + Genie), and job/pipeline clusters for generation and matching.

- **Databricks App:** the React + FastAPI web app (§6), deployed as a Databricks App.
- **Genie space** (§8) and **AI/BI dashboards** (§7) in the same workspace.
- **Notebooks:** schema/table creation and sample-data generation run as Databricks notebooks in the workspace folder `/Users/<user>/ai_parts_wizard/` (source-controlled in `code/`); executed on **serverless** compute.

12.2 Phased build

Phases are ordered by dependency; each cites the section it implements and ships its own tests (test types detailed in §13).

Phase	Deliverable	Implementations	Tests shipped
0. Foundation	Catalog + schemas, Lakebase instance, SQL warehouse, secrets/permissions in the workspace.	§3.2, §3.3, §10	Smoke: UC / Lakebase / warehouse connectivity + permissions.
1. Data model	Notebook creating the schema + Delta master tables (<code>plants</code> , <code>models</code> , <code>parts</code> wide table incl. <code>description</code> , <code>suppliers</code> , <code>supplier_catalogs</code> , <code>normalized_parts</code>) with the §3.2.1 columns.	§3.1, §3.2, §3.2.1	Data-quality: schema/constraints, FK integrity, <code>part_number</code> unique within model , blocking-key completeness.
2. Sample data	Notebook producing ~200K parts across the 8 categories + seeded ground truth (true-positive dupes, functional hard negatives, cross-material near-misses) + label table.	§4	Data-quality: row counts, category coverage, and that every seeded relationship exists and is recorded in the label table.
3. Match	Notebook (<code>build_matches.py</code>): feature build + per-category normalization → <code>gold_part_features</code> ; Spark batch matcher		Unit: <code>group_id</code> determinism, normalization, score

Phase	Deliverable	Implementations	Tests shipped
ing ✓	(offset grid-bucketing + exact L2 within (category, material) blocks — serverless blocks Spark ML LSH), stable content-derived group_id, savings, span → gold_match_groups / gold_match_group_members .	§9 , §5	conversion, savings math. Integration: match a fixture catalog end-to-end.
4. Evaluation & tuning ✓	Notebook (evaluate_matches.py) computing per-category precision/recall/F1 vs. the label table → gold_eval_metrics ; threshold tuned (L2_THRESHOLD=0.12). Result: recall 99.7% / precision ~100% / F1 99.9%.	§9 .5, §1 .4	Evaluation gate: per-category precision/recall/F1 on the label table.
5. Servicing + syncs ✓	Lakebase project ai-parts-wizard (db apw); review tables (review_dispositions , normalized_assignments); notebooks sync_facts_to_lakebase.py (Delta→Lakebase: match_groups / match_group_members / supplier_offers) and reverse_sync_review_to_delta.py (Lakebase→Delta: gold_review_dispositions , gold_normalized_parts , parts.normalized_part_id), keyed on group_id . Uses the native postgresql Spark source.	§3 .3, §5	Integration: sync round-trip (no lost review writes) — validated (two-SoR integrity).
6. Web app ✓	Databricks App ai-parts-wizard (React + FastAPI over Lakebase apw): Match Queue with the full filter set + span toggles + default view, Group Detail (members, supplier pricing w/ price-gap, disposition + normalize upserts), Genie chat stub. Deployed + UI-tested.	§6	API tests (endpoints, filter logic, default view) + React/e2e flows (filter → disposition → normalize).
7. Ge	Genie space over the gold/silver tables (domain instructions), wired into the app's	§8 ,	Genie eval: benchmark NL

Phase	Deliverable	Implementations	Tests shipped
nie + Vector Search 	docked Ask Genie panel via <code>/api/chat</code> (Genie Conversation API as the SP). Vector Search endpoint <code>apw-vs</code> + Delta Sync index <code>part_features_vs_index</code> over <code>gold_part_features</code> for online "find similar".	§9.1	questions → expected result sets.
8. Dashboards 	Lakeview dashboard — business benefit (savings by category / span, duplicate groups, review status) + engine quality (F1, coincidental, per-category table from <code>gold_eval_metrics</code>) over the all-Delta join. Published.	§7, §1.4	Query/data validation: all 6 dataset queries validated against the warehouse; no federation.

12.3 Orchestration

The nightly Databricks Job chains the steady-state sequence established in §3.3/§5: **reverse-sync (Lakebase→Delta) → matching job → facts sync (Delta→Lakebase)**, with an additional **hourly** reverse-sync for dashboard freshness. Phase 2 (generation) runs on-demand for the demo rather than nightly.

12.4 Suggested tooling

Databricks Asset Bundles (DABs) to define and deploy jobs, the app, and resources into the workspace reproducibly; notebooks/Python for generation and matching; Lakeflow for the pipeline.

13. Testing & Quality

Much of this spec's value rides on **correctness guarantees the design deliberately makes** — stable IDs, no-resurface suppression, two-SoR sync

integrity, blocking that excludes the wrong matches. The test suite exists primarily to prove those guarantees hold, in addition to normal functional coverage. Each §12 phase ships the tests noted in its row.

13.1 Test categories

- **Unit tests** — pure logic in isolation: feature normalization (per-category z-score), the content-derived `group_id` hash, L2-distance→0–1 similarity conversion, and savings computation.
- **Data-quality / expectations** — declarative checks on Delta tables: schema and constraints, FK integrity, `part_number` uniqueness *within model*, blocking-key completeness (`part_category` / `material` non-null), and validation that the §4 generator produced (and labeled) every seeded relationship.
- **Pipeline / integration tests** — the matching job on a small fixture catalog; the nightly chain (reverse-sync → match → facts-sync); the Delta↔Lakebase round-trip; and the suppression path end-to-end.
- **App / API tests** — FastAPI endpoint tests (filters, default view, disposition, normalized-number assignment) and a few React end-to-end flows.
- **Genie evaluation** — a benchmark set of natural-language questions with expected result sets, run against the curated space.
- **Model evaluation** — per-category precision/recall/F1 vs. the §4 ground truth (§9.5, §1.4). This doubles as the **promotion gate** in Phase 4.

13.2 Critical invariants (must have explicit tests)

These encode the design decisions most likely to break silently:

1. `group_id` **stability** — identical member sets across reruns yield the identical `group_id` (§3.3, §5).
2. **No resurface** — a group marked `reviewed` / `dismissed` is not re-presented after a subsequent matching run (§3.3, §5, §6.1).
3. **No join on `part_number` alone** — enforced by test data where the same `part_number` recurs across models for different parts (§3.1 join-safety rule).
4. **Blocking excludes cross-material** — identical geometry in a different material never lands in the same group (§9.2); the seeded cross-material near-misses are the fixture.
5. **Two-SoR integrity** — a review write in Lakebase always survives the reverse sync into Delta; no facts overwrite review state and vice-versa (§3.3).

13.3 Tooling & CI

- **Python:** `pytest` for unit/integration; data-quality via table constraints and/or an expectations library (e.g., DQX / Delta constraints) run as job tasks.
- **App:** `pytest` for the FastAPI layer; React Testing Library + Playwright for e2e.
- **Execution:** tests packaged and run via Databricks Asset Bundles job tasks in the target workspace; a CI workflow (GitHub Actions) runs unit + lint on PRs and triggers the bundle test job for integration/eval on merge.
- **Fixtures:** a small deterministic catalog (tens of parts, known duplicates/negatives) for fast integration runs, distinct from the full ~200K generation.

13.4 Coverage targets (illustrative)

- Unit + API: meaningful line/branch coverage on the matching and app logic (target $\approx 80\%$).
- All five §13.2 invariants: explicitly covered (non-negotiable).
- Engine quality: per-category precision/recall/F1 clear the Phase-4 targets before promotion.

14. Future Work / Optional Follow-ups

All 8 phases are built and deployed (v1.0). Optional enhancements, none blocking the demo:

1. **Vector Search "find similar" in the app.** Wire the `part_features_vs_index` (endpoint `apw-vs`) into the app so a Group Detail / part view offers live per-part nearest-neighbor lookups (§9.1). The index exists; only the app feature is unbuilt.
2. **Richer Genie tuning.** Add curated **example SQL** and **benchmark questions** to the Genie space via the UI (the builder helper can't emit sorted-id examples / answered benchmarks — §8), and consider metric views for common rollups.
3. **Productionize the Lakebase connection.** Replace the run-time plaintext OAuth-token job parameter with a **Databricks secret / the job's service-principal identity** (§10).
4. **Schedule the nightly job chain.** Wire the steady-state sequence (reverse-sync $\rightarrow$ matching $\rightarrow$ facts-sync, plus hourly reverse-sync) as a real Databricks Job / DAB (§12.3); currently the notebooks are run on demand.
5. **Managed synced tables for the forward sync.** Replace `sync_facts_to_lakebase.py` with denormalized CDF-enabled Delta gold

tables + managed Lakebase synced tables (§3.3), keeping the notebook only for the reverse direction.

6. **Per-group "ask about this group" shortcut.** Optionally add a group-scoped prompt in the Group Detail drawer that pre-seeds the global Genie chat (§6.1) — secondary to the catalog-wide chat.

Appendix A — Source

Derived from the *AI Parts Wizard — Use Case* narrative and high-level outline (Google Doc). This spec expands that outline into a product + technical specification.

Appendix B — Databricks Artifacts Inventory

Every Databricks object the solution creates, with its function. All live in the target workspace; UC objects are under `<catalog>.ai_parts_wizard`.

B.1 Unity Catalog — Delta tables

Table	Layer	Function
<code>silver_plants</code>	master	4 plants (HV0/HV1 harvesters, TR0/TR1 tractors).
<code>silver_models</code>	master	16 machine models → plant + machine type.
<code>silver_parts</code>	master	~200K parts (wide table: category, material, dims, functional discriminators, <code>description</code> , <code>model_id</code> , <code>normalized_part_id</code>). Core rationalization entity.
<code>silver_suppliers</code>	master	200 suppliers.
<code>silver_supplier_catalogs</code>	master	Per-supplier price / discount / lead-time per part.

Table	Layer	Function
gold_part_features	servin g	One canonical normalized 21-dim feature vector per part; shared by the batch matcher and the Vector Search index (CDF enabled).
gold_feature_stats	servin g	Per-category z-score normalization stats (reproducible feature build).
gold_match_groups	servin g	Match clusters: group_id , savings, span metadata, avg similarity. System of record for match facts.
gold_match_group_members	servin g	Parts per group + per-part similarity + span metadata.
gold_ground_truth	servin g	Seeded eval labels (duplicate / hard-negative / near-miss).
gold_eval_metrics	servin g	Per-category precision / recall / F1 + coincidental counts (matching-engine quality).
gold_normalized_parts	servin g	Enterprise part numbers assigned to rationalized clusters (from review).
gold_review_dispositions	servin g	Review status per group, reverse-synced from Lakebase (dashboards + suppression).

B.2 Compute

Artifact	Name / ID	Function
SQL warehouse	Serverless SQL warehouse (WAREHOUSE_ID)	Runs dashboard, Genie, and ad-hoc/eval SQL.
Serverless jobs	(one-off runs)	Execute the notebooks (generation, matching, eval, syncs).

B.3 Notebooks (`/Workspace/Users/<user>/ai_parts_wizard/` , source in `code/`)

Notebook	Function
<code>create_schema_and_tables</code>	Creates the schema + Delta master/gold tables (§3).
<code>generate_sample_data</code>	Generates ~200K parts + seeded ground truth (§4).
<code>build_matches</code>	Feature build + Spark batch matcher → match groups (§5, §9).
<code>evaluate_matches</code>	Per-category precision/recall/F1 → <code>gold_eval_metrics</code> (§4-eval, §1.4).
<code>sync_facts_to_lakebase</code>	Delta → Lakebase serving copies (§3.3).
<code>reverse_sync_review_to_delta</code>	Lakebase review state → Delta (§3.3).

B.4 Lakebase (Postgres)

Artifact	Name	Function
Project / branch / endpoint	<code>projects/ai-parts-wizard / production / primary</code>	Autoscaling Postgres serving layer.
Database	<code>apw</code>	Holds serving + review tables.
<code>review_dispositions , normalized_assignments</code>	(Lakebase SoR)	Review state written by the app.
<code>match_groups , match_group_members , supplier_offers</code>	(synced from Delta)	Low-latency read models for the app.

B.5 Application, conversational & BI

Artifact	Name / ID	Function
Databricks App	<code>ai-parts-wizard</code> (runs as its service principal)	React + FastAPI review tool: Match Queue, Group Detail, Ask Genie panel.
	"AI Parts Wizard"	

Artifact	Name / ID	Function
Genie space		Catalog-wide natural-language Q&A over the gold/silver tables; backs the app chat (§8).
Vector Search endpoint	apw-vs	Hosts the online nearest-neighbor index.
Vector Search index	part_features_vs_index	Delta Sync over gold_part_features for live "find similar" (§9.1).
Lakeview dashboard	"AI Parts Wizard — Rationalization"	Business benefit + engine-quality dashboard (§7).